Software Requirements Specification for Software Engineering: subtitle describing software

Team 18, Team Magic
Allen Tong
Amos Edeaghe
Jacob Brodersen
Joshua Engel
Nehan Munir

October 20, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
- 1.3 Stakeholders . . . 2
- 1.4 Definitions and Acronyms . . . 3
- 1.5 Characteristics of Intended Reader . . . 3

2 System Overview **3**
- 2.1 Problem Statement . . . 3
- 2.2 Product Vision . . . 4
- 2.3 System Context . . . 4
- 2.4 User Characteristics . . . 5
- 2.5 System Constraints . . . 5

3 Stakeholder Requirements **6**

4 User Personas **6**
- 4.1 Persona 1 . . . 6
- 4.2 Persona 2 . . . 7
- 4.3 Persona 3 . . . 8

5 Use Cases **9**
- 5.1 UC1: Round Setup . . . 9
- 5.2 UC2: Tasks and Answer Checking . . . 10
- 5.3 UC3: Scoring and controls . . . 12
- 5.4 UC4: Saving and Recovery . . . 13
- 5.5 UC5: 3D Visualization . . . 15
- 5.6 UC6: Guided Learning and Tutorial . . . 17
- 5.7 UC7: Create Account . . . 18
- 5.8 UC8: Daily Challenge . . . 20

6 Functional Requirements **22**
- 6.1 Formal Model . . . 22
  - 6.1.1 Game Configuration . . . 22
  - 6.1.2 Round State . . . 23
  - 6.1.3 Task Evaluation . . . 24
  - 6.1.4 Timed Scoring . . . 24
  - 6.1.5 Formal Properties . . . 25
- 6.2 Requirements . . . 25

7 Data Requirements **26**
7.1 Major Data Entities . . . 26
7.2 Data Dictionary . . . 26
7.3 Privacy and Data Retention . . . 26

8 Non-Functional Requirements **26**

9 AI and Machine Learning Requirements **32**
9.1 Model Inputs . . . 32
9.2 Model Outputs . . . 32
9.3 Evaluation Metrics . . . 32
9.4 Known Limitations . . . 32

10 Requirements Traceability Matrices **32**

11 Rationale **33**
11.1 Nonfunctional Requirements . . . 33

12 Likely Changes **34**

13 Unlikely Changes **35**

14 Appendices **36**
14.1 Appendix A: Additional Materials . . . 36
14.2 Appendix B: Development Plan . . . 36
14.3 Appendix C: Values of Auxiliary Constants . . . 36
14.4 Appendix D: Generative AI Disclosure . . . 36
14.5 Appendix E: Reflection . . . 36

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[All capstone SRS documents should have a reflection appendix. —TPLT]

[This SRS template is based on the IEEE template, with input from co-pilot and S. Smith. It will get you started. You should not modify the section headings, without first discussing the change with the course instructor. Modification means you are not following the template, which loses some of the advantage of a template, especially standardization. —TPLT]

[Feel free to change the appearance of the report by modifying the LaTeX commands. —TPLT]

[This template document assumes that a single program is being documented. If you are documenting a family of models, you should start with a commonality analysis. —TPLT]

[The SRS is not generally written, or read, sequentially. The SRS is a reference document. It is generally read in an ad hoc order, as the need arises. —TPLT]

[Guiding principles for the SRS document:

- Do not repeat the same information at the same abstraction level. If information is repeated, the repetition should be at a different abstraction level.

—TPLT]

[The template description comments should be disabled before submitting this document for grading. —TPLT]

[You can borrow any wording from the text given in the template. It is part of the template, and not considered an instance of academic integrity. Of course, you need to cite the source of the template. —TPLT]

[When the documentation is done, it should be possible to trace back to the source of every piece of information. Some information will come from external sources, like terminology. —TPLT]

[An SRS document should have the following qualities: unambiguous, consistent, complete, validatable, abstract and traceable. —TPLT]

[The overall goal of the SRS is that someone can learn, understand and verify the captured domain knowledge. They should not have to trust the authors of the SRS on any statements. They should be able to independently verify/derive every statement made. —TPLT]

1 Introduction

[Explain what system is being specified. State purpose, users, and project goals. —TPLT]

[Introduce the name of your software as Software Engineering. Using Software Engineering as a command lets you change the name if necessary and have the change propagate to all documents. —TPLT]

[Include a reference to the template this work is based on. **?** justifies the SRS structure and **?** provides the broader life-cycle context that underlines modern requirements engineering practice. —TPLT]

1.1 Purpose

[State the purpose of the system. —TPLT]

[The purpose should be in the context of the project itself, not in the context of this course. Although the "purpose" of the document is to get a grade, you should not mention this. Instead, "fake it" as if this is a real project. —TPLT]

Software Engineering is intended to solve ... [What problem does your program solve? The description here should be in the problem space, not the solution space. —TPLT]

1.2 Scope

[Briefly describe:

- Problem being solved
- Intended users
- Major functionality
- Expected benefits
- Clarifying the scope often means saying what the software doesn't do

—TPLT]

[The scope section is relevant for understanding the boundaries of the system. It is also relevant for later determining typical values of inputs. The scope should make it clear what inputs are reasonable to expect. This is a distinction between scope and context (context is a later section). Scope affects the inputs while context affects how the software will be used. —TPLT]

TODO

1.3 Stakeholders

- Customer
- End Users
- Project Sponsor
- Development Team
- Administrators

1.4 Definitions and Acronyms

[This section provides the meaning of the different words and phrases used in the domain of the problem. —TPLT]

Term	Definition
TODO	TODO

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

[This section summarizes the skills and knowledge of the readers of the SRS. It does NOT have the same purpose as the "User Characteristics" section (Section 2.4). The intended readers are the people that will read, review and maintain the SRS. They are the people that will conceivably design the software that is intended to meet the requirements. The user, on the other hand, is the person that uses the software that is built. They may never read this SRS document. Of course, the same person could be a "user" and an "intended reader." —TPLT]

[The intended reader characteristics should be written as unambiguously and as specifically as possible. Rather than say, the user should have an understanding of physics, say what kind of physics and at what level. For instance, is high school physics adequate, or should the reader have had a graduate course on advanced quantum mechanics? —TPLT]

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

Magic cubes provide an opportunity to explore numerical patterns and mathematical relationships in 3-dimensions. However, understanding these structures can become challenging due to limited sources as well as having to use static 2D images to understand a 3D pattern.

The goal of this project is to create a recreational mathematics game that allows those interested in these magic cubes as well as non standard number bases such as base 8 to further their understanding. This is planned to be a web-based game in which users will be able to complete certain challenges on a 3d representation of an 8x8 magic cube using base 8 mathematics to find patterns associated with a magic cube. This will include adding values to certain sections of the cube to complete patterns of magic cubes moving through 8 stages of difficulty while continuing to learn about these mathematical structures.

2.2 Product Vision

The vision for Magic-8 is to provide an accessible and engaging recreational mathematics application to allow interested users to further their understanding of non base 10 number systems as well as mathematical patterns found in objects such as magic cubes. The system will allow for simple interaction with the magic cube to allow users to see its many different views as well as help educate them in the patterns they will be using throughout the challenges.

This will be able to operate on supported desktop and laptop web browsers and will provide some account-based storage to track the users performance and rank them on a leaderboard. The system is planned to focus on maintainable and modular system design to potentially allow for different number bases in the future as well as ease of use for the users of the application.

2.3 System Context

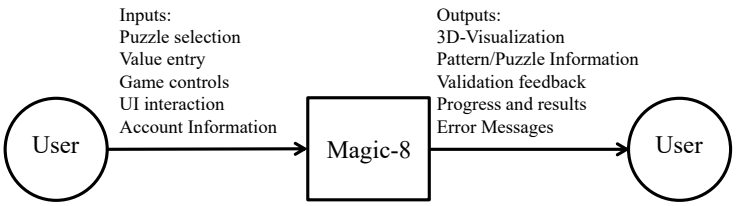


Figure 1: System Context

- User Responsibilities:
  - Provide the information required for supported account operations.
  - Select puzzles or activities and provide numerical inputs through the interface.
  - Use the available controls to manipulate the cube, select cells, and edit cell values.
  - Review validation messages and feedback provided by the application.

- Software Engineering Responsibilities:
  - Validate user inputs and identify invalid or unsupported values.
  - Process supported cube interactions and maintain the current puzzle state.
  - Present the cube and associated numerical information in the user interface.
  - Provide feedback about input validation and the outcomes of supported operations.
  - Communicate with the online service to authenticate users and save or retrieve designated user data.
  - Inform users when account or storage operations fail or cannot be completed.
- Web Browser Responsibilities:
  - Execute the delivered application and provide the environment in which its interface is displayed.
  - Render the two-dimensional interface and three-dimensional cube visualization.
  - Relay user interactions to the application.

Magic-8 is intended for recreational and educational exploration. This means it will not be operating in a highly safety or mission-critical enviornment. The application provides a simple way to interact with slighlty more obscure mathematical ideas rather than any critical operations.

2.4 User Characteristics

The intended users are individuals who are interested in recreational mathematics, numerical puzzles and structures, and non standard number bases. Users may range from beginners who may have just started learning base 8 and never encountered magic cubes or vice versa, to experienced mathematical enthusiasts interested in trying a new challenge.

Users are expected to have basic computer skills, including the ability to navigate a web browser, use pointer-based controls, and enter numerical values. No programming knowledge is required for use. Only basic mathematic understanding of addition and subtraction is expected as the application can help to teach the subjects it covers.

2.5 System Constraints

Magic-8 shall be designed to run within desktop and laptop web browsers without requiring an installation. The functionality will initially be developed in python as well as supported python libraries such as pygame. It will then be translated into webassembly code for execution in browser. This means that all python code must be translatable with the chosen tool into webassembly including anything used for the user interface of the application.

The system shall also support online account functionality, requiring network communication with the selected service for remote data operations. The specific service, supported web browsers, and rendering technologies are yet to be established. This will happen as implementation decisions are finalized.

3 Stakeholder Requirements

SR1 This program shall validate the stakeholder's answers in terms of both position and octal answer.

SR2 This program shall visualize the current octal magic cube being solved by the stakeholder.

SR3 This program shall teach stakeholders how octal number works.

SR4 This program shall teach stakeholders how magic squares and then cubes work.

SR5 This program shall verify if the stakeholders satisfied all the requirements of a level.

SR6 This program shall show stakeholders different cubes with tasks of the same difficulty at the same level.

SR7 This program shall time stakeholders and show them their current and final score.

SR8 This program shall allow stakeholders to access it deployed on a free web-hosted website.

4 User Personas

4.1 Persona 1

- Name: Scotty Thompson
- Age: 16
- Occupation: Student
- Location: Toronto, Canada
- Description: Scotty is a Grade 10 student who stumbled upon this program when he was learning octal numbers in his Computer Technology class. His teacher took a glance at the program and recommended using it to study for the octal/hexadecimal test. The only understanding he has of the octal system is from his computer technology course, which he pays attentions to. but is lost in his train of thoughts when learning about concepts that don't apply to everyday life.

- Goals:
  - With a basic undertanding of octal numbers, Scotty wishes to learn how to do them faster (like a multiplication table), so he doesn't get stuck on test questions requiring addition/subtraction.
  - He doesn't want a simple math checker, he wants more information on why he got the answer wrong, alongside a visual demonstration on how to get the correct answer.
- Frustrations:
  - He doesn't care about the "magic" and intrinsicality of the magic cube, as he feels like he has no use of learning about these patterns.
  - He will get bogged down if shown literature about magic cubes as he is using this program for only one goal.
- Technical Ability: He is a gamer who has played browser games since he was a child and is able to quickly click and type.

4.2 Persona 2

- Name: John Fye
- Age: 73
- Occupation: Retired (Former Stenographer)
- Location: Hamilton, Canada
- Description: John is a former court stenographer who recently got a kick out of learning about John Hendricks's work. He wishes to test out his ability to see the patterns in magic cubes as he has been doing base-10 magic square puzzles in his puzzle book for the last 20 years.
- Goals:
  - He wants to learn more about octal numbers, as he has a very basic understanding about it, but a lot of Mr. Hendricks's work finds patterns in different bases, and he feels like learning about octal will help him appreciate Mr. Hendricks's accomplishements, such as order 16 perfect magic tesseract.
  - He wants to discover the patterns in the cube, such as how you can use a starting number to find other numbers (such as the complimentary).
  - He really wants to dive deep into
- Frustrations:

  - The only thing he currently knows about octal is how it cycles from 0 to 7, he never did any math with it.
  - He struggles to look at a screen for a long time and is usually zoomed in. He takes extensive breaks, which might lead him to forgetting where he left off and what was his train of thoughts at that point.

- Technical Ability: He was a court stenographer, which allows him to type fast. However, he never really used a computer before, which will be a problem when he is using a mouse. He also doesn't want to look at a mostly white screen when using the program.

4.3 Persona 3

- Name: Charles Dean
- Age: 23
- Occupation: Masters in English
- Location: Halifax, Canada
- Description: Charles is someone that starts off their day playing the "Dles", which is a collection of single-level browser games that refreshes daily, like Wordle. He is tired of the flood of general knowledge trivia games, and wants to play a game that tests his brain without needing to know stuff beforehand.
- Goals:
  - He wants a daily game that he can complete as part of his daily rotation of games (shouldn't be too long).
  - He wants the game to run on his browser with minimal load times, not really interested in getting stuck due to the 3D visualization.
- Frustrations:
  - He isn't really interested in math, so might lose interest if the puzzles are too complex.
  - He needs a hook (either visually or with flair) that ensures that he plays it more than once as he doesn't start off interested in the subject matter.
- Technical Ability: He is very familiar with browser games and while he hasn't done math since high school, he does remember an inkling of what hexadecimal is from his high school computer course.

5 Use Cases

5.1 UC1: Round Setup

Primary Actor

User

Preconditions

- Browser game is deployed on an accessible website
- Game has access to database containing cube configurations
- No rounds is currently in progress

Main Success Scenario

1. User presses START
2. System wipes screen clean and pings cube database wih appropiate query
3. Cube database sends the values of the cells alongside with a cube number (as an identifier)
4. System creates the 3D visualization and discretely populates visualization cells with its correct, hidden values
5. System prompts user to choose the scoring system (decimal or octal)
6. User clicks on the scoring system they want and its confirmation
7. System selects the appropiate scoring system
8. System randomly populates a single visualization cell with its correct octal and has a final popup describing the next level

Alternative Flows

- If Cube database doesn't ping back, the System will already have one default cube formation stored locally in its code, so that cube formation will be used instead. The System will continuing pinging the database every 30 seconds to see if the database is active and notifiy its developers the first time it doesn't get an answer.
- If the 3D visualization fails, either fall back to a simpler visualization method or show an error.
- If the scoring system fails, continue with the steps after displaying a message about how scores isn't going to be tracked for this round.

Postconditions

- All errors and log-type messages will appear as notifications starting from the bottom on a corner.
- There is a black 3D visualization of a cube in the centre.
- There is a notification showing '512 of 512 cells loaded' and there is a single octal number on the cube.
- On one of the upper corners, there is a scoring component consisting of an unstarted timer and a score integer.

5.2 UC2: Tasks and Answer Checking

Primary Actor

User

Preconditions

- The system should be set up according to the postconditions of UC1

Main Success Scenario

1. User chooses to begin the levels by clicking START.
2. System displays a textbox asking the user to select the complement of the single populated cell
3. User moves the 3D visualization using the chosen movement method in order to locate the complement
4. User clicks on the cell representing the complement, thereby selecting it
5. System flashes white and displays the properties of the intial populated cell and a textbox for the answer
6. User enters in the complement's value in the Addition/Subtraction Box
7. System alerts the scoring syetem by sending a query with the level of the task that was just finished
8. System shows the user a level overview with a difficulty slider
9. User chooses difficulty
10. System populates the cells according to the difficulty selected and displas a START button
11. User clicks START and selects the cells that corresponds to the half-orthogonal for the starting cell

12. User clicks the DONE button
13. System displays the Addition/Subtraction Box for selected cells
14. User enters in the octal values and clcks DONE
15. System queries the scoring system with information of the level just completed
16. Previous steps repeat for the remaining steps: orthogonal, diagonal on continuous plane, diagonal not on continuous plane, another diagonal not on continuous plane, triagonal, split triagonal
17. System asks scoring sytem for final score
18. Scoring sstem calculates final score
19. System ends after displaying a final score and buttons to navigate to other sections

Alternative Flows

- User is able to pull up level overview when they are in the level by selecting an arrow poking out of the side if they are stuck
- If the User clicks DONE after selecting the wrong cells, the game will flash red and deselect the selected squares, and siplays a message saying they selected the wrongs squares
- When the user inputs the wrong answer in the Addition/Subtracton box, the box will flash red and clear the incorrect answer, letting them try again. The System will also notify the scoring system that an incorrect answer has been given.
- If the 3D visualization is stalling (e. due to the user running out of memory), the System shows a toggle to switch to the 2D view of the cube
- If the user is signed in, the are able to save the score if they want to

Postconditions

- System should display the final score front and centre
- There should be an option to either create a new account (while being able to save the current score), sign in, or view account information
- There are options displayed to either start a new run, go to the tutorial, or enter other modes

5.3 UC3: Scoring and controls

Primary Actor

System

Preconditions

- The system is currently active and in a viable state of execution
- The scoring system is an instance which is volatile and doesn't keep track of information
- The scoring system is well integrated with near-instantaneous signals from the system
- The scoring system is able to use other APIs to ensure that its time signals is correct

Main Success Scenario

1. System queries the scoring system with level number
2. Scoring system initializes a stopwatch, error counter and correct scoring function
3. System pings the scoring system, telling them that the level has started
4. Scoring system starts the timer and listens for signals indicating errors or completion
5. System pings for an error
6. Scoring system increments error counter
7. System pings for the completion of the level
8. Scoring system runs appropiate scoring function with time and error counter as inputs
9. Scoring system returns the score and its instance terminates

Alternative Flows

- If the user selected a different scoring base or level, the system (not scoring system) will add the corresponding values for each addition:

Level	Decimal	Octal
1	10	14
2	20	30
3 and 4	30	44
5 and 6	40	60
7 and 8	60	110

- If the user is able to enter in additional numbers, they are rewarded 10 points for desimal and 14 points for octal.

- If the user clicks STOP/EXIT, the scoring system instance is terminated without any return
- If the user clicks PAUSE, the stopwatch in the scoring system will pause and keep track of the time paused (so it doesn't mess up when checked with eternal APIs), but the scoring system instance will continue running

Postconditions

- The score given can be verified as base score * (1350/seconds) + score for correct cells/octals for an individual level
- The system's internal final score is incrememented between levels
- At the end, the system shall show the internal final score (verified as last internal score + level 8 score)

5.4 UC4: Saving and Recovery

Primary Actor

User

Preconditions

- System should be in a valid state of execution
- User database should be active (in the sense that it exists and connectable) and triggerable
- The connection between the database and system should be closed.

Main Success Scenario

1. User selects cells and pressed DONE
2. System saves which cells were selected
3. System verifies that the correct cells were selected
4. System saves that the correct cells were selected
5. System calculated how much points the user gets for the selection
6. System saves the number of points
7. System displays the Addition/Subtraction box based on saved game state
8. User completes level

9. System caculates and saves final score
10. User chooses to save the score
11. System pings user database with user information
12. User database pings back (saying that it is active)
13. System shares final score with user database
14. User database saves score
15. System closes the connection with the user database
16. User clicks to return to main menu
17. System refreshes its information via user database
18. System displays the main menu with user information (including latest/highest final score)

Alternative Flows

- If a save fails (either internal or with a database), the system will retry 3 times. If the system still fails to save, it will notify the user and warn that any further progress might not be saved. The user is free to keep playing and the system will retry saving either every 30 seconds or when the user reaches the net save point (whichever comes first).
- If the program crashes, after exhausting all possible recovery options, the system will save the last recoverable state in the browser cache. It will then recommend the user refresh or close the browser page. The next time the user opens the page, it will ask the user if it wants to load the previous progress or start anew.

Postconditions

- The connection between the user database and system should be closed
- The schema and integrity of the user database should be equivalent to its precondition
- The system shouldn't have any saved data about internal levels (such as level scores and cell selection) after finishing all the levels
- The browser cache would optimall always be empty unless there has been a crash and the user hasn't returned to the game

5.5 UC5: 3D Visualization

Primary Actor

System

Preconditions

- System is in a valid state of execution
- 3D visualization is a plugin (removing it just causes the system to use another visualization)
- 3D visualization doesn't have any data inputted into it (it starts anew everytime a cube is selected)

Main Success Scenario

1. System gets the cube values and number from the cube database
2. 3D visualization starts to initialize in the middle of the screen
3. System chooses a starting position
4. 3D visualization filled with the correct octal number at that position
5. System repositions the cube to the centre of the screen
6. 3D visualization centers the model at the filled cell
7. User moves mouse across a face of the cube
8. 3D visualization rotates accordingly across its face due to mouse
9. User types in a coordinate in a navigator bar
10. System repositions the cube to the centre of the screen
11. 3D visualization centers the model at that coordinate
12. User presses down on the "s" or "down" key on the keyboard
13. 3D visualization rotates down accordingly across its face due to keyboard
14. User drags mouse on an area off the cube
15. System repositions towards the mouse (so that the cube is off centre)
16. User scrolls down on the mousewheel
17. 3D visualization moves along the z-axis, so that the second and next layers of the cube is exposed

18. User selects a cell
19. System saves which cell has been selected and checks its correctness
20. System tells the 3D visualization which cell has been selected and what colour it should be
21. 3D visualization keeps the cell and previous cell as a bright white light source
22. User enters in number in Addition/Subtraction box
23. System checks correctness of the octals inputted
24. System tells the 3D visualization what to project onto the cell's surface
25. 3D visualization shows the correct octals the user inputted
26. System calculates level score
27. 3D visualization keeps filled cells

Alternative Flows

- If the user types in a coordinate that would belong on another layer, the system will reposition the cube to the centre of the screen and the 3D visualization centres on the selected xy coordinates. It will then move through each layer until it arrives at the correct z value.
- If the user is selecting the wrong cell (ex. as part of a triagonal), the system will inform the 3D visualization that the cell chosen is wrong. The 3D visualization will make the cell chosen red.
- If the user is on a non-face and tries to rotate the cube, the user will be able to see the missing layers either behind or on the R/LHS of the camera, but they will be transparent so the person is aware of where they are located in the cube.
- If the system is calculating final scores, the 3D visualization isn't needed, so that instance of the 3D visualization plugin is terminated.

Postconditions

- The 3D plugin shouldn't have any data associated with it (such as information about which cells are filled)

5.6 UC6: Guided Learning and Tutorial

Primary Actor

User

Preconditions

- The system is in a valid state of execution
- The system is not in the process of conducting a level/game

Main Success Scenario

1. User selects the tutorial option from the menu
2. System clears screen and projects empty cube
3. System chooses a cube and projects all cells
4. User clicks button to learn about octal
5. System displays popup with an octal explanation, solved examples and calculator
6. User clicks "Test" button
7. System generates random questions and textboxes for solutions
8. User enters solutions
9. System verifies answers and closes popup
10. User clicks button to learn about magic squares
11. System removes navigation controls and centres on a face (thereby a magic cube)
12. System popups explanation about magic square
13. User clicks "Test" button
14. System shows a stored magic square
15. User enters solutions
16. System verifies and shows a harder magic square
17. User exits back to main menu
18. User starts first game
19. System sets up game
20. User clicks "Hint" button

21. System popups coordinate hint about where the next cell is located
22. User clicks "Hint" button
23. System traces path to the correct square with explanations
24. User selects correct cells
25. System displays the Additon/Subtraction Box
26. User clicks "Hint" button
27. System displays the octal steps to arrive at the correct answer

Alternative Flows

- If the user asks for more magic square examples, the system displays a link to an educational website.
- If the user taps the stopwatch, the system will display a message allowing the use to disable the scoring system. If the user wants to, it will result in no scores being recorded.
- If the user runs out the level timer (22.5 minutes), the system will display a messgae saying that the score wouldn't be counted. The system will not record the score and allow unlimited mistakes and hints.

Postconditions

- The system is in a valid state of execution

5.7 UC7: Create Account

Primary Actor

User

Preconditions

- The system is in a valid state of execution
- The system is not in the process of conducting a level (can sign in between levels)
- User database should be active (in the sense that it exists and connectable) and triggerable
- The connection between the database and system should be closed.

Main Success Scenario

1. User clicks button to sign in
2. System saves current game state in browser cache and displays information to sign in
3. User selects "Create Account"
4. System displays a page with a username textbox and username guide
5. User types in appropiate username
6. System verifies username follows username requirements and pings user database
7. User database verifies username doesn't exist
8. Systems displays a page with a password textbox and password guide
9. System verifies password follows password requirements and creates a new user profile and inserts it into the user database
10. System asks user to log in
11. User logs in
12. User database confirms user
13. System recreates game state via browser cache and deletes cache
14. User completes game
15. System saves final score, time needed and date by appending it in user database
16. User database saves and closes connection
17. User goes to main menu
18. System opens connection to user database and displays information
19. User closes browser tab
20. System closes connection to user database

Alternative Flows

- If the user writes a username or password that doesn't follow the requirements, the system will display an error message alongside an option to type in another name.
- If a user database ping or save fails, the system will retry 3 times. If the system still fails to get a response to the ping, it will notify the user and say that they failed in making an account. It will also warn that any further progress might not be saved. When the user finishes the final level, the system will retry pinging the user database and if it responds, the system will display a message saying that the user creation page is working again.

Postconditions

- The connection between the user database and system should be closed
- The schema and integrity of the user database should be equivalent to its precondition

5.8 UC8: Daily Challenge

Primary Actor

User

Preconditions

- The system is in a valid state of execution
- The system is not in the process of conducting a level/game
- User and Daily Challenge database (DC) should be active (in the sense that it exists and connectable) and triggerable
- The connection between the databases and system should be closed.

Main Success Scenario

1. User clicks button to begin Daily Challenge
2. System asks if the user wants to log in
3. User logs in
4. System pings DC
5. DC replies with Daily Challenge plane colour, orientation, and starter
6. System saves Daily Challenge game state in browser
7. System asks if the user is ready to play
8. User starts challenge
9. System uses cube and starter given instead of pinging cube database and random starter
10. User finishes game
11. System pings user database and DC
12. System saves final score, time needed and date by appending it in DC column in user database

13. System creates a copyable graphic that can be displayed by the user
14. User database saves and closes connection
15. DC sends leaderboard scores to System
16. System checks if the user score belongs on leaderboard
17. System saves final score and time needed at appropiate rank by inserting it into DC's daily leaderboard column
18. System closes connection to DC
19. User goes to main menu
20. System opens connection to user database and displays information
21. User goes to Daily Challenge leaderboard
22. System pings DC for leaderboard data
23. System displays leaderboard with copy button
24. User clicks out
25. System closes DC's connection
26. User closes browser tab
27. System closes connection to user database

Alternative Flows

- If a DC ping or save fails, the system will retry 3 times. If the system still fails to get a response to the ping, it will notify the user and say that their Daily Challenge attempt will not be counted. Since the user already saw the puzzle, they will not be able to redo the Daily Challenge until it refreshes.

Postconditions

- The connection between the databases and system should be closed
- The schema and integrity of the bases should be equivalent to its precondition

6 Functional Requirements

6.1 Formal Model

This formal model describes the configuration of a game round, the evaluation of tasks, and the calculation of the users score. It defines the conditions under which a task is considered complete and the properties that the scoring system must maintain. The model supports the functional requirements concerning task execution, answer evaluation, and score calculation (FR7–FR50).

6.1.1 Game Configuration

Let the set of tasks in a game round be

$$\mathcal{T} = \{T_1, T_2, \ldots, T_8\}.$$

Each task T_i has an associated target condition and a basic score value. Let:

$$B_i = B(T_i), \qquad B_i \in \mathbb{N}_0.$$

Where B_i is the basic score awarded for correctly completing task T_i. The target conditions are defined by the game configuration and are evaluated against the current puzzle state.

The octal target values used by the game include

$$S_1 = 1001_8, \qquad S_2 = 2002_8, \qquad S_3 = 4004_8.$$

These values are interpreted as octal numbers. For example,

$$1001_8 = 513_{10}, \qquad 2002_8 = 1026_{10}, \qquad 4004_8 = 2052_{10}.$$

These values are used by the application to define their target conditions. For instance,

$$\theta_i(C) = \begin{cases} 1, & \text{if } L_i(C) = s_i, \\ 0, & \text{otherwise.} \end{cases}$$

where $L_i(C)$ is the sum of relevant cells, and

$$\theta_i(C) = \begin{cases} 1, & \text{if condition met} \\ 0, & \text{otherwise.} \end{cases}$$

is the target condition.

6.1.2 Round State

A game round is represented by the state

$$R = (C, i, A, P),$$

where:

- **Puzzle Configuration (C):** The current state of the magic cube, represented by a three-dimensional array of cell values:

  $$C \in V^{n \times n \times n},$$

  where n is the dimension of the cube and V is the set of permitted cell values, including any designated empty value used for unfilled cells. Each element $C_{x,y,z}$ represents the value of the cell at position (x, y, z), for $1 \leq x, y, z \leq n$.

- **Current Task Index (i):** The index identifying the task currently being attempted:

  $$i \in \{1, 2, \ldots, 8\}.$$

  The value of i determines which task's completion conditions are evaluated against the current configuration C.

- **Attempt Set (A):** The set of task attempts recorded during the round:

  $$A \subseteq \{(j, C_j, t_j, q_j) \mid j \in \{1, \ldots, 8\}\},$$

  where j identifies the task, C_j is the cube configuration submitted for evaluation, t_j is the elapsed time associated with the attempt, and $q_j \in \{0, 1\}$ indicates whether the attempt was unsuccessful (0) or successful (1). Each recorded attempt therefore contains sufficient information to identify the task, the evaluated configuration, its elapsed time, and its result.

- **Accumulated Score (P):** The total score awarded to the player during the round:

  $$P \in \mathbb{N}_0, \qquad P = \sum_{j=1}^{8} W_j,$$

  where W_j is the score awarded for task T_j, and $\mathbb{N}_0$ denotes the non-negative integers. A task contributes to P only when it has been completed successfully and its score has not already been awarded.

The round state must remain consistent after every accepted state transition. In particular, the accumulated score P must agree with the recorded task rewards, and unsuccessful attempts must not increase the score.

The current task index satisfies

$$1 \leq i \leq 8.$$

For each task T_i, let $E_i(C)$ be the evaluation function that returns true when the puzzle configuration C satisfies the task's required conditions and false otherwise:

$$E_i(C) = \begin{cases} 1, & \text{if task } T_i \text{ is satisfied,} \\ 0, & \text{otherwise.} \end{cases}$$

An attempt is considered successful only when the corresponding evaluation function returns 1. Changes to the puzzle state must be evaluated against the requirements of the current task.

6.1.3 Task Evaluation

Let a_i represent an attempt to complete task T_i. The result of evaluating the attempt is

$$\text{Result}(a_i) = E_i(C).$$

A successful attempt satisfies the task's target condition. An unsuccessful attempt does not satisfy that condition and must not be treated as a completed task.

The task completion indicator is defined as

$$Q_i = \begin{cases} 1, & \text{if task } T_i \text{ has been completed correctly,} \\ 0, & \text{otherwise.} \end{cases}$$

This acts as a persistent tracker of completion even after the next task begins changing $E_i(C)$.

For a round containing eight tasks, the number of completed tasks is

$$N_{\text{completed}} = \sum_{i=1}^{8} Q_i.$$

Consequently,

$$0 \leq N_{\text{completed}} \leq 8.$$

A task can contribute to the completed-task count only after its correct completion has been established by the evaluation function.

6.1.4 Timed Scoring

Let B_i be the basic score for task T_i, and let t_i be the elapsed time, in seconds, associated with its successful completion. For $t_i > 0$, the timed score is calculated as

$$\text{TimedScore}_i = \text{round}\left(\frac{B_i \cdot 1350}{t_i}\right).$$

Here, round denotes rounding to the nearest integer. The scoring rule rewards correct completion while taking the time required to complete the task into account.

The score awarded for task T_i is defined as

$$W_i = \begin{cases} \text{TimedScore}_i, & \text{if } Q_i = 1 \text{ and the score has not been awarded,} \\ 0, & \text{otherwise.} \end{cases}$$

The total score for a round is therefore

$$P = \sum_{i=1}^{8} W_i.$$

This definition ensures that the total score is the sum of the individual task scores and that an unsuccessful attempt does not contribute points. A completed task must not be awarded points more than once within the same round.

6.1.5 Formal Properties

The following properties describe expected behaviour of the task evaluation and scoring system.

Theorem 1: Correctness of Task Completion. For any task T_i, the task is considered successfully completed only if its evaluation function returns true:

$$Q_i = 1 \implies E_i(C) = 1.$$

Justification. By definition, task completion depends on satisfying the task's required conditions. An attempt that fails evaluation cannot be recorded as a successful completion.

Theorem 2: No Score for an Unsuccessful Attempt. For any task T_i,

$$Q_i = 0 \implies W_i = 0.$$

6.2 Requirements

[Functional requirements describe what the system should do. They are typically expressed as a set of conditions or capabilities that the system must have. These requirements are usually derived from the stakeholder requirements and use cases. —TPLT]

[Use SHALL statements. Requirements must be atomic, testable, and traceable. —TPLT]

FR1: The system shall ...

FR2: The system shall ...

FR3: The system shall ...

[The requirements are given labels so that they can be referenced in other parts of the SRS. The label names should be meaningful mnemonics, not numbers. —TPLT]

7 Data Requirements

[The Data Requirements section describes what information the system must manage. It includes the major data entities, their attributes, and the relationships between them. It also includes a data dictionary that defines the meaning of each attribute. —TPLT]

7.1 Major Data Entities

[What information must exist for the system to accomplish its goals? —TPLT]
[For each entity, provide a short explanation. —TPLT]

- User
- Song
- Session
- Report

7.2 Data Dictionary

[For each entity, define all important attributes. —TPLT]

Attribute	Type	Description
user_id	UUID	Unique identifier
email	String	User email address

Table 2: Data Dictionary

7.3 Privacy and Data Retention

[Discuss storage, retention, deletion, privacy controls, consent, encryption, and compliance requirements. —TPLT]
TODO

8 Non-Functional Requirements

NFR1: **Cube Manipulation Performance**

The system shall update the visible cube orientation within 100 milliseconds of receiving a supported manipulation input.

Rationale: Delayed manipulation feedback can make the three-dimensional cube feel unresponsive and make it difficult for users to understand the relationship between their input and the resulting orientation.

Fit Criterion: Under the defined reference test environment, the visible cube orientation updates within 100 milliseconds for at least 95% of measured supported rotation inputs.

Traces: FR??.

Stakeholder Requirements: SR2.

NFR2: **Cell-Value Editing Performance**

The system shall visibly reflect an accepted cell-value edit within 100 milliseconds of the user submitting the edit.

Rationale: Prompt feedback helps users understand whether their changes have been accepted and reduces uncertainty when solving a puzzle.

Fit Criterion: Under the defined reference test environment, at least 95% of accepted cell-value edits are visibly reflected within 100 milliseconds.

Traces: FR??.

Stakeholder Requirements: SR1.

NFR3: **Puzzle Opening Usability**

A first-time user shall be able to open a puzzle without external assistance or prior training.

Rationale: Users should be able to begin solving a puzzle without needing to consult instructions or receive help from another person.

Fit Criterion: At least 80% of participants in a usability evaluation shall successfully open a puzzle without external assistance or prior training.

Traces: FR??.

Stakeholder Requirements: SR4.

NFR4: **Cube Manipulation Usability**

A first-time user shall be able to manipulate the three-dimensional cube using the supported rotation controls without external assistance.

Rationale: The cube is the primary visual representation of the puzzle. Users need to understand how to change its orientation to inspect and interact with its cells.

Fit Criterion: At least 80% of participants in a usability evaluation shall successfully perform each defined cube-rotation task without external assistance.

Traces: FR??.

Stakeholder Requirements: SR2.

NFR5: **Cell Selection Usability**

A first-time user shall be able to select an individual puzzle cell using the supported selection controls without external assistance.

Rationale: Users must be able to identify the cell they intend to inspect or edit. Unclear selection behaviour may cause unintended changes to the puzzle.

Fit Criterion: At least 80% of participants in a usability evaluation shall correctly select the requested cell in each defined cell-selection task without external assistance.

Traces: FR??.

Stakeholder Requirements: SR2.

NFR6: **Cell-Value Editing Usability**

A first-time user shall be able to enter and submit a valid value for a selected puzzle cell without external assistance.

Rationale: Editing cell values is a core interaction in solving the puzzle. The editing process should be understandable without requiring users to learn a complex sequence of actions.

Fit Criterion: At least 80% of participants in a usability evaluation shall successfully enter and submit a valid value for a selected cell without external assistance.

Traces: FR??.

Stakeholder Requirements: SR2.

NFR7: **Separation of System Components**

The system shall organize mathematical logic, user-interface logic, three-dimensional visualization logic, and online storage interactions into identifiable components with documented responsibilities.

Rationale: Separating these responsibilities reduces the risk that changes to one part of the system will unintentionally affect unrelated functionality.

Fit Criterion: A code inspection shall identify separate components responsible for each of the four specified areas, with documented responsibilities for each component.

Stakeholder Requirements: SR6.

NFR8: **Independent Testing of Mathematical Logic**

The system shall allow its core mathematical operations to be tested independently of the user interface and three-dimensional visualization.

Rationale: Independent testing makes it easier to identify mathematical errors without requiring the complete application to be executed for every test.

Fit Criterion: The automated test suite for core mathematical operations shall execute without launching the user interface or rendering the three-dimensional cube.

Stakeholder Requirements: SR1.

NFR9: **Browser Compatibility**

The system shall operate in every desktop and laptop web browser included in its defined support list.

Rationale: Users should be able to access the game through supported browsers without requiring a specific browser outside the stated compatibility requirements.

Fit Criterion: The defined core functional test suite shall pass on every browser and browser version included in the system's documented support list.

Stakeholder Requirements: SR8.

NFR10: **Browser-Based Deployment**

The system shall allow users to access and operate the game through a supported web browser without requiring installation of Python or a standalone desktop application.

Rationale: Browser-based access reduces setup effort and allows users to run the game on supported desktop and laptop devices without installing a separate application.

Fit Criterion: On a clean reference device, a tester shall be able to launch and use the game through a supported browser without installing Python or a standalone desktop application.

Stakeholder Requirements: SR8.

NFR11: **Puzzle-State Integrity**

The system shall maintain a valid in-memory puzzle state following supported cube manipulations, cell selections, and accepted cell-value edits.

Rationale: The puzzle state must remain consistent throughout interaction so that users do not encounter unintended changes or false puzzle representations.

Fit Criterion: Automated tests shall verify that the puzzle state satisfies the defined state-validity rules after every state update in the test suite.

Traces: FR??.

Stakeholder Requirements: SR5.

NFR12: **Invalid Input Handling**

The system shall reject invalid cell-value inputs without terminating the application or leaving the puzzle state invalid.

Rationale: Users may enter unsupported values or make mistakes while editing cells. Invalid inputs should not corrupt the puzzle or interrupt continued use of the game.

Fit Criterion: Automated tests shall submit every defined category of invalid cell-value input and verify that the input is rejected, the puzzle state remains valid, and the application continues operating.

Traces: FR??.

Stakeholder Requirements: SR1.

NFR13: **Online Operation Failure Handling**

When an online storage operation fails, the system shall inform the user that the operation was unsuccessful.

Rationale: Users need accurate feedback about whether their data has been stored or retrieved successfully. Misleading feedback may cause users to believe that their progress or puzzle data has been saved when it has not.

Fit Criterion: Tests that simulate each defined online storage failure shall verify that the system displays an appropriate failure indication.

Traces: FR??.

Stakeholder Requirements: SR8.

NFR14: **Network Communication Security**

All network communication involving account credentials or user data shall use HTTPS with valid TLS.

Rationale: Encrypting network communication helps protect credentials and user data against interception or unauthorized modification during transmission.

Fit Criterion: Network inspection shall confirm that all requests involving account credentials or user data use HTTPS with valid TLS, and that the system does not transmit such information through unencrypted HTTP requests.

Traces: FR??.

Stakeholder Requirements: SR8.

NFR15: **User Data Isolation**

The system shall prevent an authenticated user from accessing another user's private stored data through its supported interfaces and operations.

Rationale: Puzzle data, saved progress, and other private information associated with an account should remain accessible only to authorized users.

Fit Criterion: Authorization tests shall verify that an authenticated test account cannot retrieve, modify, or delete private data belonging to another test account through any supported operation.

Traces: FR??.

Stakeholder Requirements: SR8.

NFR16: **Authentication Secret Protection**

Application-controlled persistent storage shall not store authentication secrets in plaintext.

Rationale: Storing authentication secrets in plaintext may expose user accounts if stored data is accessed without authorization.

Fit Criterion: Inspection of application-controlled persistent storage shall find no authentication secrets stored in plaintext.

Traces: FR??.

Stakeholder Requirements: SR8.

NFR17: **Personal Data Minimization**

The system shall collect and retain only the personal information necessary to provide its implemented functionality.

Rationale: Limiting the collection and retention of personal information reduces privacy risks and the amount of data that must be protected.

Fit Criterion: A review of the system's data fields, collection operations, and persistent storage shall confirm that every collected or retained personal information field has a documented functional purpose.

Traces: FR??.

Stakeholder Requirements: SR8.

NFR18: **Keyboard Accessibility**

All essential puzzle controls shall be operable using a keyboard without requiring pointer interaction.

Rationale: Keyboard operation supports users who cannot use a mouse or other pointer-based input device and provides an alternative way to interact with essential puzzle functionality.

Fit Criterion: A keyboard-only test shall verify that users can access and operate every essential puzzle control, including puzzle navigation, cell selection, and cell-value editing, without pointer interaction.

Traces: FR??.

Stakeholder Requirements: SR8.

NFR19: **Visual Contrast**

Essential interface text and controls shall meet a minimum contrast ratio of 3:1 against their adjacent background.

Rationale: Adequate contrast helps users distinguish interface elements and read essential information.

Fit Criterion: An automated or manual contrast assessment shall confirm that every essential interface text element and control meets the specified minimum contrast ratio of 3:1 against its adjacent background.

Traces: FR??.

Stakeholder Requirements: SR8.

9 AI and Machine Learning Requirements

[Refer to Dr. Mosser's guest lecture on AI requirements. —TPLT]
[Include only when applicable. —TPLT]

9.1 Model Inputs

TODO

9.2 Model Outputs

TODO

9.3 Evaluation Metrics

[Accuracy, Precision, Recall, F1, BLEU, WER, etc. —TPLT]
TODO

9.4 Known Limitations

[Explicitly document expected failure modes. —TPLT]
TODO

10 Requirements Traceability Matrices

[The Requirements Traceability Matrix (RTM) is a table that shows the relationship between different types of requirements. It is used to ensure that all requirements are traceable and that no requirements are missed. The typical flow is: Stakeholder Need -¿ Use Case -¿ Requirement —TPLT]
[There is also a traceability matrix between functional and nonfunctional requirements. The purpose of some FRs is to satisfy NFRs. Therefore, it is important to show the relationship between the two. —TPLT]
[Further traceability to test cases and modules will be given in other documents. —TPLT]

Stakeholder Req	Use Case	Functional Req
SR??	UC1	FR1

Table 3: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR1	NFR??

Table 4: Traceability Between Functional and Nonfunctional Requirements

11 Rationale

[Provide a rationale for the decisions made in the documentation. Rationale should be provided for scope decisions, functional requirements, nonfunctioanl requirements, data requirements, and AI requirements. You don't need a rationale for everything, ust the major decisions that are not obvious. You should have traceability to the entities in the document that you are referencing. —TPLT]

11.1 Nonfunctional Requirements

- **Performance (NFR1, NFR2):**

  Cube manipulation and cell-value editing are two of the most frequent interactions in the magic cube game. These operations need to provide prompt feedback so that users can understand the effects of their actions without unnecessary delays.

- **Usability (NFR3, NFR4, NFR5, NFR6):**

  The usability requirements focus on the primary tasks that users must perform to interact with the puzzle successfully. These tasks represent the essential interaction workflow. Requiring at least 80% of usability evaluation participants to complete these tasks without external assistance provides a measurable baseline for whether the interface is understandable to its intended users or not.

- **Maintainability (NFR7, NFR8):**

  Separating mathematical operations, user-interface behaviour, 3D visualization, and online storage into identifiable components reduces coupling between unrelated responsibilities. This decision allows individual components to be modified with less risk of affecting other parts of the system.

- **Portability (NFR9, NFR10):**

  The system is intended to be accessed through desktop and laptop web browsers rather than a platform-specific desktop application. Browser-based deployment reduces installation requirements and allows users to access the game through supported environments without installing Python or additional standalone software.

- **Reliability (NFR11, NFR12, NFR13):**

  The correctness of the puzzle state is essential because subsequent calculations, visual updates, and user interactions depend on it. Separately defining requirements for valid state maintenance, invalid input handling, and online failure handling ensures that the system can respond predictably to both expected operations and error conditions.

- **Security and Privacy (NFR14, NFR15, NFR16, NFR17):**

  Because the system may support user accounts and online storage, appropriate measures are needed to protect user information. Encrypted network communication protects data during transmission, while access restrictions prevent users from accessing other accounts' private data.

- **Accessibility (NFR18, NFR19):**

  The system should provide alternative ways for users to interact with and understand the puzzle. Keyboard-operable controls support users who cannot rely on pointer-based interaction, while sufficient visual contrast helps make interface elements distinguishable.

12 Likely Changes

LC1: [Give the likely changes, with traceability to the entity that is likely to change. — TPLT]

(NFR1, NFR2) : If it is found that the 100ms response times are unreasonable can easily be changed to more fitting ones.

(NFR19) : May need to change the given standard for contrast if it is not considered enough.

(UC3) : The team still aren't really aware of how they want the scoring mechanism to work as timing doesn't feel like the funnest scoring method. So far, they followed Dr. Rapoport's idea, but this might change after testing from stakeholders. Also, a lot of the work the system does can be offloaded to the scoring system, as the scoring system is just a glorified time signal in this use case.

(UC4) : There might be too much saving involved in this, as the internal object representing the game in the system is saved before being changed on the front end (which the user sees) every single time. It might be useful to use the browser cache so data is saved only at major steps.

(UC5) : The team still hasn't started learning about 3D visualization and how it translates to a browser, so extensive QA is needed to arrive at an approachable cube visualization.

(UC8) : There are multiple ideas on how the Daily Challenge is going to be implemented, and since it is a strech goal and the team doesn't really know what aspects of their project is "fun", it can be a vastly different implementation on release.

13 Unlikely Changes

LC2: [Give the unlikely changes. The design can assume that the changes listed will not occur. —TPLT]

(NFR18) : Will not likely be lessening our goal to allow those without the ability to use a mouse to be able to interact with the program.

(NFR9, NFR10) : The web based approach was one of the first comments made by Dr. Rapoport and so unless there is no other option we would like to continue with this method.

(UC1, UC2, UC7) : These describe core aspects of the magic 8 project, so they are very unlikely to change as there were numerous meetings to arrive at these. Obviously, the rules of how magic cubes and octal is not going to change, so the use cases here are locked for now. Account creation is unlikely to change as numerous components depend on it, so changing its workflow would require significant changes throughout the codebase.

(UC6) : Whilst this might not be optimal way to teach the subject matters related to our project, it is the best the team currently came up with. Also, as it is a stretch goal, there isn't going to be a lot of focus on it until the principle project is finished.

14 Appendices

14.1 Appendix A: Additional Materials

TODO

14.2 Appendix B: Development Plan

[This section is optional. It is used to explain the plan for developing the software. In particular, this section gives a list of the order in which the requirements will be implemented. In the context of a course this is where you can indicate which requirements will be implemented as part of the course, and which will be "faked" as future work. This section can be organized as a prioritized list of requirements, or it could show the requirements that will be implemented for "phase 1", "phase 2", etc. —TPLT]

14.3 Appendix C: Values of Auxiliary Constants

[Show the values of the symbolic parameters introduced in the report. —TPLT]

[The definition of the requirements will likely call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. —TPLT]

[The value of FRACTION, for the Maintainability NFR would be given here. —TPLT]

TODO

14.4 Appendix D: Generative AI Disclosure

[Provide full transparency on how generative AI was used in the creation of this document. This includes any text, figures, or tables that were generated by AI. If any skills were created for AI agents to review the documentation, those should be disclosed here. —TPLT]

For sections 2, 8, and 6.1 there was AI used for text generation using a particular format from unformatted text due to my unfamiliarity with LaTeX.

14.5 Appendix E: Reflection

[This isn't part of the SRS, but it is part of the course. —TPLT]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you

to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Team Answers

Questions 5 and 6 have been answered as a team.

5. There are a few key pieces of knowledge and skills that the team will collectively need to acquire to successfully complete this capstone project. First, we must further our understanding of magic cubes and the patterns that are present within them so that we can accurately create the game which Dr. Rapoport has asked us to create. Second, we must learn how to do 3D projection onto a 2D plane to allow for us to display a cube using only pygame which is required for our current plan to use a python to WebAssembly translation method. Third, we will need to learn data storage methods and how to effectively send and retrieve data that may need to be encrypted for privacy.

6. Josh will be taking on the task of learning 3D projection on a 2D plane. The two ways in which he will go about doing this are using a template version of the code for a simple object and studying it to gain further understanding of the concepts as well as watching instructional videos on the topic from online sources such as youtube. He

mainly plans to focus on breaking down the code since that will be sure to work with pygame while some videos that are found can be witholding crucial information.

Jacob will be completing the task of furthering the team's understanding of magic cubes and their patterns. The first method Jacob will use is discussions with the supervisor, Dr. Rapoport. Dr. Rapoport is already very knowledgeable on magic cubes and their properties as well as the context of the project. As such, Dr. Rapoport will be a very useful source of information. The second method Jacob will use is literature review. Magic cubes and their properties are well documented in academic literature as well as other online sources (e.g., websites). Jacob will use this literature review to supplement knowledge gained from discussions with Dr. Rapoport.

Josh Engel

1. One thing that went well with writing this deliverable is compared to the last deliverable we were more organized on what we wished to complete and when, as well as starting sooner allowing for easier communication throughout the group. I also used the provided materials from earlier years as a guide more than with the last deliverable which helped to identify what exactly I should be doing.

2. The main pain point I found throughout this deliverable was how to make NFRs verifiable. I was unsure as to what would be considered acceptable for fit critereon and so changed formatting and explanations a few seperate times leading to uneeded time spent. Eventualy I came up with a very simple solution which was to reach out and "poll" my aquaintances as well as teammates on what they deemed would be acceptable and proceeded from there.

3. Personally I worked on the Non-functional Requirements as well as the formal model. For the NFRs I based them off earlier conversations we had with our supervisor Dr. Rapoport as well as getting feedback from him for changes after the initial draft. For the formal model I used the functional requirements determined by my peers as well as had a peer review of it to help reach a state the group was happy with.

4. Multiple courses I have taken will be of great help to us throughout this project. A major one of these courses is 4HC3: Human and Computer Interfaces, this will help in designing the applications UI in a human forward fashion which allows for better usability this is especially important for me since I am currently slated to be working on front end development where this is important.

Nehan Munir

1. One thing that went well with writing this deliverable is that the development plan helped us focus in on an idea that we can make quantifieable steps for. Speaking with Dr. Rapoport over Reading Week helped ensure us that we were following Dr.

Rapoport's vision (as he is a stakeholder for this project). This made it easier to split up work as we all knew that everyone was on the same page from the offset, while we needed numerous meetings beforehand to land on a stable idea for the Capstone project.

2. Whilst the previous section might make it seem like there wasn't really any conflict on the idea, we still needed a level of connectivity between every section in the project. For example, we needed to be clear how stakeholder requirements lead to the FRs and NFRs connect, as well as the connection between use cases and the functional requirements. Getting the appropiate hyperlinks was difficult without knowing exactly what the other person wrote and for which point. This was solved by essentially finishing our work before meeting with Dr. Rapoport, allowing us ample time to reread everyone's work and see the connectivity of ideas.

3. As I worked on the stakeholder requirements, I came up with 8 solid ones based on a file that Dr. Rapoport wrote explaining how magic squares work and how he wants the project to work. I was then able to ask him to look them over during our Reading Week meeting. There, we realized that we didn't explain our strech goals to him (ex. tutorials on octal and magic squares), and we were able to get them approved, thus allowing us to add it as stakeholder requirements.

4. I will be working on formalizing how magic squares will work (the rules) within the system. I feel like SFWRENG 2DM3/2FA3 - Discrete Mathematics with Applications I-II and Linear Algebra I will be important. Discrete Math allows me to ensure that the cube is always in a steady state and map out all of the possibilites the cube can take. Linear Algebra will help as we are likely going to represent all 512 values inside the cube as a matrix, and being able to create formal rules on how the matrix can be manipulated is important (whether it is a database check to ensure all the cube matrixes are actually magic cubes, or an instaneous check on what the user is currently entering).

Jacob Brodersen

1. I did not write any of the sections for this deliverable (I focused on the Hazard Analysis). However, the crossover of the two documents (referencing the requirements from the Hazard Analysis) went well because the group kept in communication to ensure the two documents were consistent.

2. I did not write any of the sections for this deliverable (I focused on the Hazard Analysis). However, one pain point was trying to figure out how to incorporate LO_SpecMath into this document. We had difficulty deciding the format and location of the content. To resolve this, Josh (the writer of the formalization section) and I discussed various ideas. We ended up deciding upon a method suggested in the Discussion page on Avenue because Dr. Smith commented that he liked the team's approach.

3. Most of our functional requirements came from our supervisor, Dr. Rapoport. In terms of the other types of requirements (i.e., non-functional requirements, safety and security requirements, etc.), these were brainstormed internally.

4. I think the SFWRENG 3RA3 has been helpful already in this course. Stating the obvious, it was helpful for writing the requirements in the Hazard Analysis and SRS documents. Furthermore, the 3RA3 course knowledge was helpful in the Problem Statement document for defining the problem.

   I think that SFWRENG 4HC3 will become helpful when we start thinking about interface design; in this course, we did a full interface design for our own app idea. This is very similar for what we will have to do in this capstone.

   In SFWRENG 3A04 we were able to design a software architecture and develop the code for a classification application. This will help us in the design and development phases of this capstone.

   Finally, SFWRENG 2AA4 gave us practical experience in software design and development, which will help in this capstone when creating the application.